

Martingale Wall Simulator — Architecture

Document status: design baseline · **Date:** 25 August 2026 · **Applies to:** prototype release

This document describes how the Martingale Wall Simulator is built and why. It is written for two readers at once. Sections 1–3 and 8 are for anyone who needs to understand what the system does and what state it is in. Sections 4–7 are for the engineers building and maintaining it. Technical terms used anywhere are defined in the glossary at the end.

1. What the system does

The simulator answers a single question: **following a given staking plan, how many consecutive losing trades can an account absorb before it can no longer place the next required trade?**

That breaking point is called the **wall**. The application exists to locate it precisely, and to explain clearly why it is unavoidable.

The reasoning is arithmetic, not opinion. On a trade that pays back less than the amount staked, a win returns only a fraction of what was risked. Recovering a run of losses therefore requires staking more than the run cost — and the amount needed grows faster than the losing run itself. Every recovery-staking pattern meets a wall. The only variables are where it sits and how quickly it arrives.

What the system deliberately does not do. It does not predict markets, does not estimate the probability of a losing run, and does not present the staking plan as profitable. It assumes the worst case — every trade loses — and reports the consequence. Presenting it as anything else would misrepresent the arithmetic it exists to demonstrate.

2. Who uses it, and how

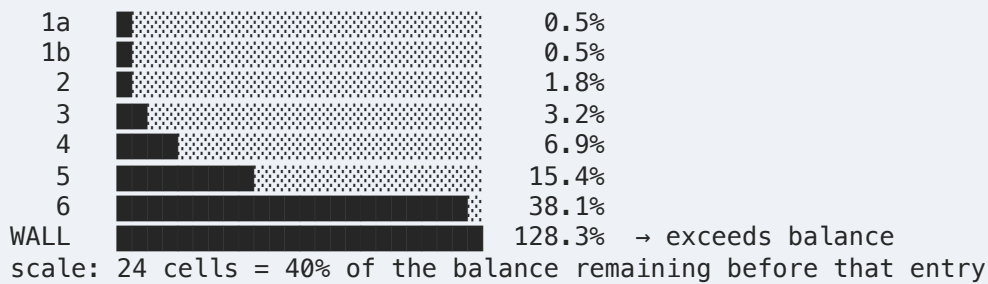
A single visitor, using a browser, with no account and no login. Three steps:

1. **Read the theory** on the landing page — the payout mechanic, why recovery staking has a ceiling, and a FAQ explaining how to read the results.
2. **Run a simulation** from three inputs: starting capital, payout percentage, and the second entry size. Four further parameters sit under an *Advanced* panel at sensible defaults.
3. **Read the results** — every entry in the ladder, the stake it required, what remained afterwards, and the point at which the account could no longer continue.

Every run is stored, so results can be revisited and compared. Stored runs can be cleared, and clearing is reversible — see section 6.

3. How the escalation behaves

The reason the wall arrives suddenly is that each entry consumes a larger share of what is left. Using the reference case — \$1,000 capital, \$5 and \$5 on the first candle, \$18 on the second, 92% payout — the share of the remaining balance each entry demands:

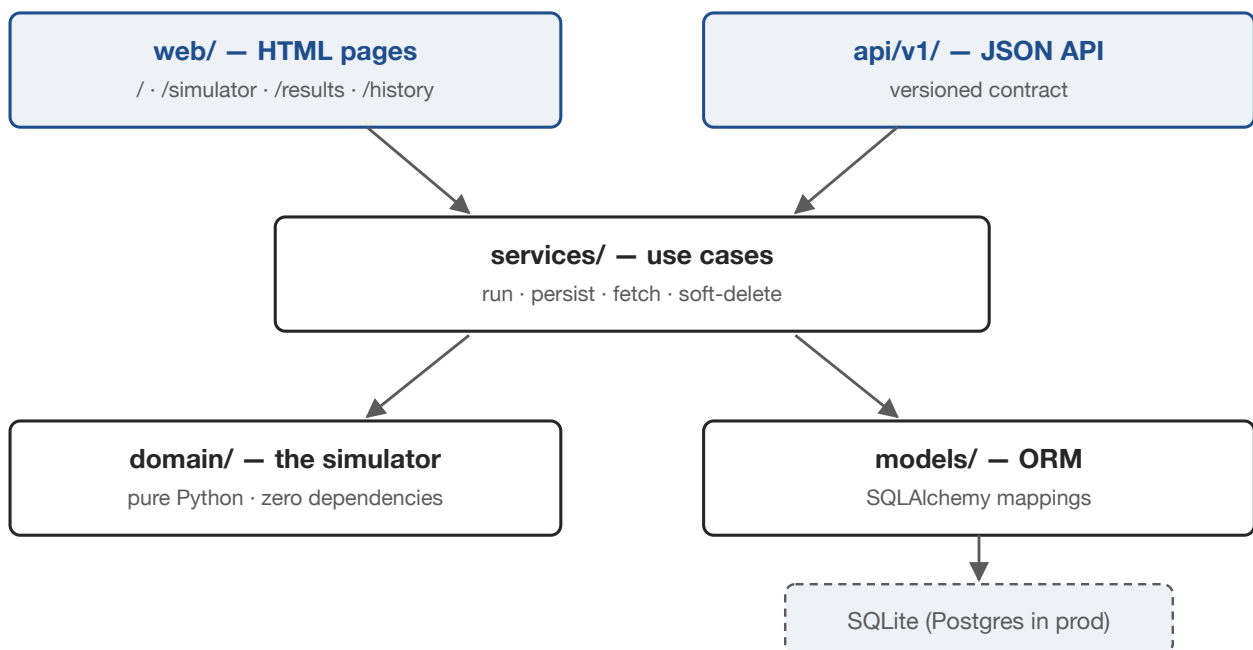


The first five entries look harmless — none risks more than 7% of what remains. The sixth demands 38%. The seventh demands more than the account holds, and the plan stops. Nothing warns the trader in advance, which is precisely why the tool exists: **the danger is invisible until the entry before last.**

4. System structure

The application is a single deployable unit. It serves web pages and a JSON interface from the same process, with no separate frontend build.

It is organised in layers. Each layer may depend only on the layer beneath it, never the reverse.

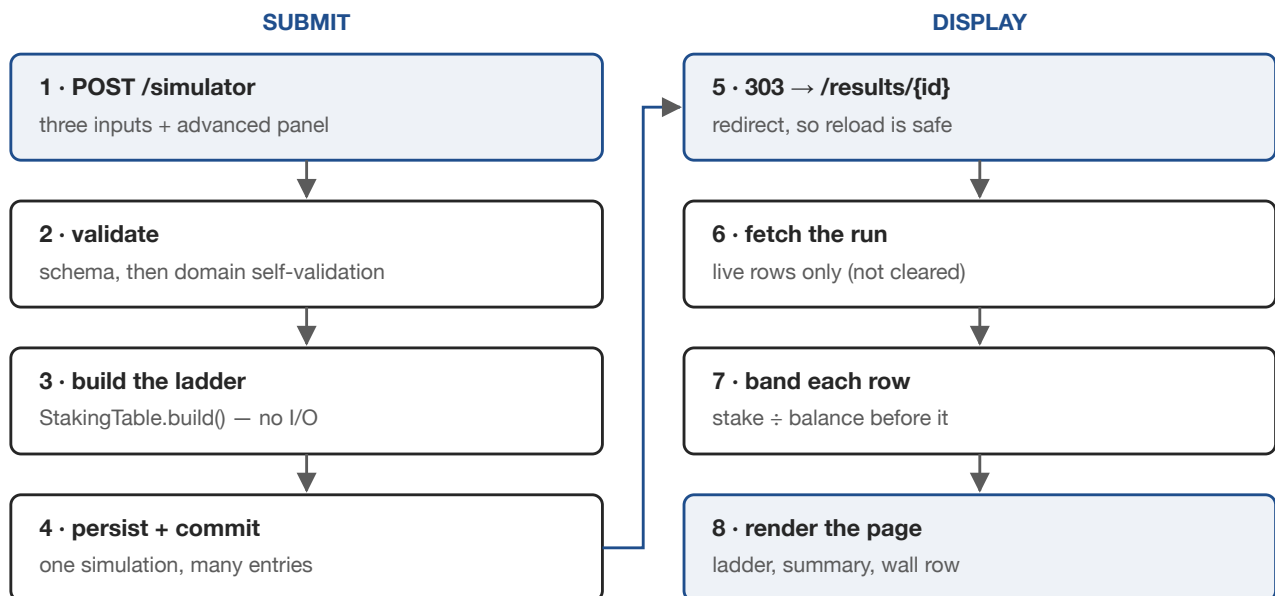


The rule that matters most: the domain layer — the simulation itself — depends on nothing. It imports no web framework, no validation library, and no database library. It runs under a bare Python interpreter with nothing installed.

This is not architectural decoration. It means the calculation can be tested without starting a server, reused from a command line or a batch analysis, and left untouched when the web framework is upgraded. The simulator is the asset; everything else is replaceable packaging around it.

Two HTTP surfaces, versioned differently. The JSON API is versioned (`/api/v1/...`) because outside consumers depend on its shape and must not break when it changes. The web pages are not versioned, because their only consumer is the browser being served and their addresses are things people bookmark.

5. What happens when a simulation runs

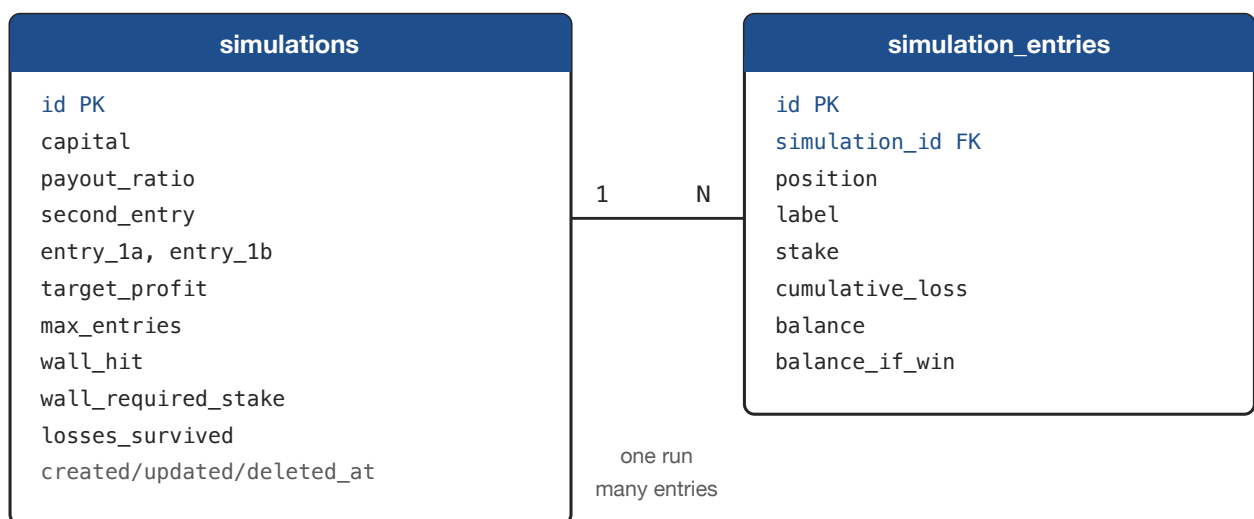


Two details in that flow are deliberate.

Validation happens twice, and that is correct. The web schema rejects malformed input — a missing field, text where a number belongs. The domain then applies its own rules: capital must be positive, the payout must fall between 0 and 100%, the target profit cannot be negative. The domain's checks are the authoritative ones because they protect the calculation from every caller, not only from the web form. A single translation point converts a domain rejection into a readable error response, so no individual route handles it.

Submitting redirects rather than rendering. The result page is reached by a fresh request, so reloading it re-reads a stored run instead of re-submitting the form.

6. What is stored



Two tables rather than one. The entry ladder could have been stored as a single structured blob against each run. It is a separate table instead, because blob storage behaves differently across SQLite, PostgreSQL, and MySQL — it would be the one part of the schema needing rework at the production

move, which is exactly what section 7 exists to avoid.

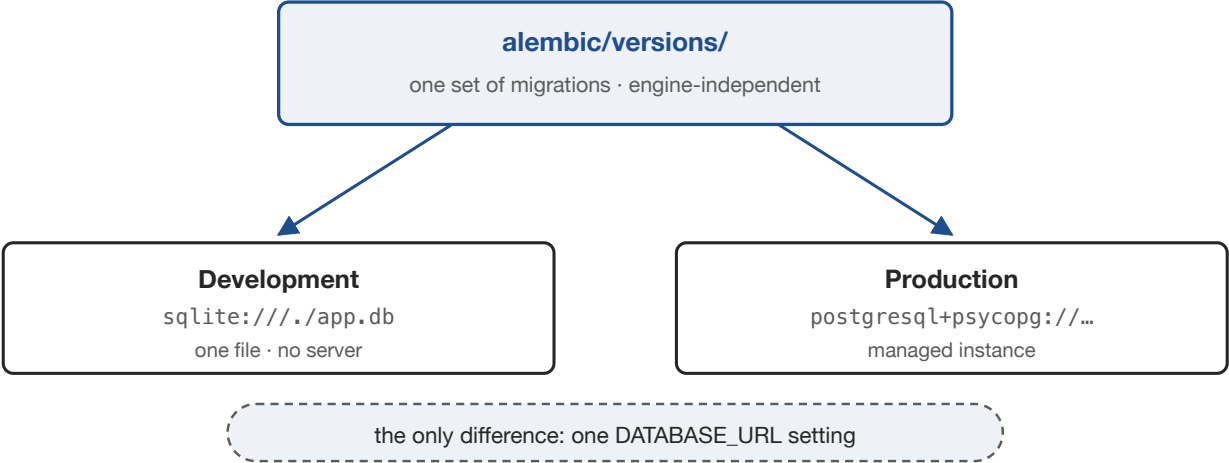
Timestamps. Every row records when it was created and last changed, in UTC with an explicit time zone. The values are set by the application rather than the database, because the three database engines each generate timestamps with different precision and different time-zone handling. Setting them in one place makes all three agree.

Clearing is reversible. Deleting a run does not remove it. It records the time of deletion in a `deleted_at` field, and every ordinary query ignores rows that carry one. The data remains for inspection and can be restored by clearing that field. "Clear the table" marks every live run as deleted; it never discards anything.

The correctness of this rests on a single rule: **every read path goes through one shared query helper that applies the filter.** A query written by hand that omits it would silently show deleted runs, and that is the one way this pattern fails.

7. Moving to production

The prototype runs on SQLite — a single file, no server, no setup. Production will run on PostgreSQL or MySQL. The schema is written so that this is a configuration change.



The same migration files build the schema on either engine. Moving is three steps: point the database setting at the new server, run the migrations, and copy the existing rows if they are worth keeping.

What makes this work is discipline in the schema rather than anything clever at migration time — no engine-specific column types, no structured blobs, explicit lengths on text columns, every constraint named, and no reliance on the accidental row ordering SQLite happens to provide. Each of those is a thing SQLite tolerates and PostgreSQL does not.

Known limitation. SQLite has weak support for altering existing columns, so a future migration that changes a column type will need care. This is a limitation of the prototype database, not of the migration tooling, and it disappears once production moves to PostgreSQL.

8. How we know it works

Check	What it protects
Domain test suite	The arithmetic — both wall outcomes, every rejected input
Reference case	The published example still produces \$589 needed, \$459 left, 7 survived
Independence check	The simulator still runs with no libraries installed

Check	What it protects
Interface tests	Correct status codes, correct shapes, readable errors
Deletion test	A cleared run is retained in storage and absent from listings
Style and type checks	Consistent formatting, no type errors
Accessibility pass	Contrast, keyboard navigation, colour never the only signal

Every check runs identically whether the developer works directly on their machine or inside a container. That equivalence is deliberate: a defect that appears in one and not the other means the two environments have diverged, and that divergence is then the defect to fix.

Glossary

Alembic — the tool that records and applies changes to the database structure, so every environment can be brought to the same shape reliably.

Binary option — a trade with two outcomes: a fixed payout if the prediction is right, the loss of the stake if it is wrong.

Candle — one fixed interval of price movement, the unit of time an entry is placed against.

Entry — a single trade placed as part of the staking plan.

Ladder — the full sequence of entries a plan produces, from the first to the wall.

Migration — one recorded change to the database structure, applied in order with all others.

ORM — the layer that maps stored rows onto objects in the code, so the application is not written in raw database language.

Payout ratio — the proportion of the stake returned as profit on a winning trade. At 92%, a \$100 winning stake returns \$92 profit. Because this is below 100%, recovery staking has a ceiling.

Recovery stake — the amount that, if it won, would recover every loss in the run so far.

Schema — the structure of the stored data: which tables exist and what each column holds.

Soft delete — marking a record as deleted while keeping it stored, so the action is reversible and the data remains available for inspection.

SQLite / PostgreSQL / MySQL — database engines. SQLite is a single file requiring no server, suited to a prototype; the other two are server-based and suited to production.

Stake — the amount of money risked on one entry.

Wall — the point at which the next required stake exceeds the money remaining, so the plan cannot continue. The subject of this application.